

RESEARCH

RiskawareTrader: A Reinforcement Learning based Portfolio Optimization for Risk Averter

Min Yang · Jin Wang · Yi Hu

Received: 19 June 2025 / Revised: 20 October 2025 / Accepted: 30 November 2025

© The Author(s) 2025

Abstract

Portfolio optimization is a crucial decision-making component of intelligent financial system. It aims to maximize profits while managing risks by strategically allocating individual investments. Risk averter usually prefer the preservation of capital to the potential for a higher-than-average return, thus challenging existing portfolio optimization methods. This paper proposes a risk aware portfolio optimization method named RiskawareTrader for risk averter by incorporating downside risk, which could efficiently characterize losses. Specifically, this paper models the portfolio optimization as a Markov Decision Process, and adopts reinforcement learning method to get the optimum portfolio. RiskawareTrader adopts the downside risk in the reward function to penalize portfolios with high potential risks. In order to validate the performance of RiskawareTrader, this paper launches numerical experiments with some public datasets, namely the Dow Jones Industrial Average (DJIA) and the China Securities Index 300 (CSI300), in terms of annual return, maximum drawdown and Sharpe ratio. As demonstrated by the results, RiskawareTrader could achieve higher returns and better risk management under various market conditions compared to some baseline models.

Keywords Portfolio optimization · Reinforcement learning · Downside risk

1 Introduction

Portfolio management is a key component in financial market. The objective is to construct an investment portfolio that maximizes returns while minimizing risks. Portfolio optimization aims to reduce potential risks through diversification strategies while simultaneously maximizing long-term returns[1], usually involving dynamic distributing funds across a diversified set of assets, i.e., stocks, bonds, and cash equivalents. The process is inherently complex due to influences from economic growth, inflation, interest rates, and monetary policies. Effective management requires investors to possess strong financial expertise and deep market insight for flexible strategy adjustment. However, psychological biases, emotional interference, and misinterpretation of market information often lead investors to make irrational decisions[2], exacerbating portfolio risk exposure and undermining long-term returns. Thus, portfolio optimization remains a challenging task, influenced both by market dynamics and investor behavior.



Quantitative trading is a mature approach to portfolio management. It uses computational technologies and big data to capture market trends, so it has greater objectivity than traditional methods based on experience[3]. Recent advances in deep learning (DL) and reinforcement learning (RL) have transformed this field. DL excels in extracting multilevel features from financial data[4], while RL optimizes trading strategies dynamically to maximize long-term returns[5, 6]. The integration of DL's feature representation and RL's adaptive decision-making offers a robust framework for balancing risk and reward in portfolio optimization.

However, existing quantitative trading methods face a critical limitation, they overlook the unique needs of risk averter who prioritize capital preservation over high returns. Such investors exhibit heightened sensitivity to losses, preferring stable returns over volatile opportunities. Traditional reinforcement learning strategies, while effective for general portfolio optimization, treat risk as an auxiliary constraint rather than integrating it into core decision-making. For example, they often fail to dynamically weight risk and return based on market regimes, leading to suboptimal performance in bear markets. This gap creates a pressing need for strategies that explicitly address downside risk, a challenge that is not addressed by current approaches.

This paper addresses the above gap by designing the RiskawareTrader Strategy, namely the RTS, a deep reinforcement learning framework tailored for risk averter. Specifically, RTS integrates a downside risk evaluation mechanism into the reward function, using the investor's target return as a benchmark. This mechanism penalizes returns below the target with a penalty, guiding the model to prioritize risk control. Additionally, RTS introduces a novel feature fusion network combining Multi-Scale CNN (MSCNN), UNet architecture, Multi-Dimensional Attention (MDA), and MLP. MSCNN captures multi-level market features, UNet retains contextual dependencies via skip connections, MDA enhances focus on critical signals, and MLP maps features to portfolio weights.

This paper makes three key contributions:

- In response to the unique needs of risk-aware investors, this study creatively introduces a downside risk evaluation mechanism in the optimization strategy. This mechanism, centered around the investor's target return rate, precisely rewards or penalizes the portfolio's actual performance, effectively guiding the model to be more cautious in risk control while pursuing returns.
- This study innovatively designs an advanced feature fusion extraction algorithm by cleverly integrating MSCNN, UNet architecture, MDA and MLP. This algorithm not only deeply mines multi-dimensional and multi-level information in financial market data but also significantly enhances the model's sensitivity to market dynamics and predictive accuracy.
- Through empirical research in different market environments, the model's outstanding performance in balancing risk and reward has been verified, providing a reliable investment strategy for risk-aware investors.

The remainder of this paper is organized as follows. Section 2 reviews related work on portfolio optimization and reinforcement learning. Section 3 introduces the preliminaries, including the problem formulation and the TD3 algorithm. Section 4 elaborates on the proposed models and methods. Section 5 presents the experimental results and the performance analysis. Section 6 discusses the setting of hyperparameters. Finally, Sect. 7 discusses the conclusions and future research directions.

2 Related Work

Portfolio optimization research has primarily evolved along two theoretical lines: mean-variance analysis and capital growth theory. Markowitz's seminal mean-variance model[1] balances risk and return within single-period horizons, while capital growth theory emphasizes long-term wealth accumulation via semi-logoptimal strategies[7]. Traditional methods, including momentum strategies[8], contrarian models[9] and nonparametric techniques[10], rely on heuristic feature engineering, but these approaches are constrained by rules defined by humans, limiting their ability to capture nonlinear dynamics of modern markets.

Reinforcement learning (RL) has transformed portfolio optimization by framing it as a Markov Decision Process (MDP). Early frameworks like Q-learning[11] and Deep Q-Networks (DQN)[12] laid the foundation for stochastic decision-making, though their discrete action spaces are ill-suited for continuous weight optimization. Policy-based methods like Deep Deterministic Policy Gradient (DDPG) address this limitation. Jiang et al.[13] integrated ensemble evaluators into DDPG for cryptocurrency portfolios, and Xiong et al.[14] validated its adaptability across market cycles. However, these approaches often overlook risk-aware objectives, focusing on return maximization without explicit downside protection.

Recent studies have sought to integrate risk constraints into RL frameworks. Enkhsaikhan et al.[15] proposed a risk-averse RL model using Bayesian Neural Networks (BNN) to quantify aleatoric and epistemic uncertainty. In a related vein, Enkhsaikhan and Jo[16] introduced an Augmented Lagrangian Multiplier (ALM) technique within a Constrained MDP (CMDP) framework, ensuring no constraint violations during testing and outperforming traditional risk-averse RL algorithms in volatile markets. Other works, such as Cui et al.[17] incorporating CVaR for tail risk control and Yoo et al.[18] proposing safety-first policies, treat risk as an auxiliary constraint rather than integrating it into core decision-making, limiting their suitability for risk averter. Chen et al. [19] introduced an RL framework that dynamically adjusts risk preference parameters in the mean–variance model, leveraging historical market data to optimize portfolio weights.

Deep learning-based feature extraction has advanced portfolio optimization. Li et al.[20] combined long-sequence modeling and cross-asset attention, Zou et al.[21] used cascaded LSTMs for temporal features, and Xu et al.[22] introduced relation-aware transformers. However, few studies integrate multi-scale feature fusion for risk-aware decision-making. Ramya and Suresha [23] used K-means clustering for stock selection and combined LSTM and RL for weight allocation.

This study addresses these limitations by embedding risk awareness directly into the reward function via a target return benchmark, ensuring proportional penalties for downside deviations. Unlike prior work that relegates risk management to post-hoc adjustments, RTS's architecture features a multi-scale CNN-UNet attention network that learns to prioritize risk-return tradeoffs from raw market data. This dual focus on adaptive feature extraction and risk-sensitive reward design offers a unified framework for dynamic portfolio optimization in uncertain markets.

3 Preliminaries

In the context of applying deep reinforcement learning to portfolio optimization, the investor is referred to as the agent. At each time step t , the agent selects an investment action $a_t \in A$ contingent on the current portfolio state $s_t \in S$. Following this decision, the market environment responds with a reward signal $r_t(s_t, a_t)$, while the portfolio evolves into a new state s_{t+1} influenced by both market dynamics and the chosen action. The overarching goal is to optimize a decision-making policy that maximizes the expected long-term cumulative reward, balancing return enhancement and risk mitigation[24].

Model-based reinforcement learning algorithms rely on explicit state transition models, requiring knowledge or estimation of the transition probability $P(s'|s, a)$, the likelihood of reaching state s' given current state s and action a . In portfolio analysis, backtesting with historical data often assumes that trading actions do not alter future asset states[7]. This implies fixed transition probabilities regardless of investment decisions, treating investor actions as non-disruptive to market dynamics, though such assumptions may not hold in real-world financial environments.

To address these limitations, this study employs the Twin Delayed Deep Deterministic Policy Gradient (TD3), a model-free RL algorithm well-suited for continuous action spaces. TD3 builds upon the Deep Deterministic Policy Gradient (DDPG) framework, introducing architectural and algorithmic enhancements to improve stability and convergence. Its twin-network structure, comprising a policy network and a target network, mitigates learning instabilities. By delaying policy network updates relative to the critic networks, TD3 further stabilizes

training. Exploration noise mechanisms enable efficient exploration-exploitation trade-offs, making TD3 particularly effective for high-dimensional portfolio optimization tasks.

TD3 addresses the overestimation bias common in DDPG through a dual-critic architecture. It maintains two critic networks $Q_1(s, a|\theta^{Q_1})$ and $Q_2(s, a|\theta^{Q_2})$, along with their corresponding target networks $Q'_1(s, a|\theta^{Q'_1})$ and $Q'_2(s, a|\theta^{Q'_2})$. The target Q-value calculation selects the minimum output from the two critics, as shown in the Eq.(1).

$$y_i = r_i + \gamma \min \left(Q'_1(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'}), \theta^{Q'_1}), Q'_2(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'}), \theta^{Q'_2}) \right) \quad (1)$$

Here, $Q_1(s, a|\theta^{Q_1})$ and $Q_2(s, a|\theta^{Q_2})$ are critic networks, $Q'_1(s, a|\theta^{Q'_1})$ and $Q'_2(s, a|\theta^{Q'_2})$ are the respective target critic networks.

These mechanisms enable TD3 to converge to optimal policies in complex environments, reducing training variability and ensuring reliable performance.

Our implementation integrates TD3 into the RiskawareTrader Strategy (RTS) for portfolio optimization. As detailed in Algorithm 1, the process involves sampling initial states, generating actions, and storing state-action-reward transitions in a replay buffer for iterative training. Through successive updates, the algorithm refines policies and value functions, ultimately converging to an optimal portfolio allocation strategy.

Input: Number of iterations M , number of stages T , number of trajectories K , batch size N , learning rate η , discount factor γ , tolerance ϵ

Output: Optimal parameters for networks UNet(U), MSCNN(C), MDA(A), MLP(M), optimal policy, and optimal value function $J(\theta)$

```

1 Initialize networks:  $U, C, A, M$ , and target networks  $U', C', A', M'$ ;
2 for  $m = 1$  to  $M$  do
3   for  $t = 1$  to  $T - 1$  do
4     for each trajectory  $k = 1, 2, \dots, K$  do
5       Compute the action  $a_k^t$  using the current networks, State feature
          $f_k^t = C(s_k^t)$ , state embedding  $e_k^t = U(f_k^t)$ , and action
          $a_k^t = M(A(e_k^t))$ ;
6       Obtain the next state  $s_k^{t+1}$ , store the transition  $(s_k^t, a_k^t, s_k^{t+1})$ ;
7     end
8   end
9   Select a batch  $B$  of  $N$  trajectories;
10  for each trajectory in the batch do
11    Compute reward function  $\hat{r}(s_k^t, a_k^t, s_k^{t+1})$  for  $t = 1, 2, \dots, T$ ;
12    Approximate action-value function  $Q^\theta(s_k^t, a_k^t)$ ;
13    Compute target  $y_k = \frac{Q^\theta(s_k^t, a_k^t) + L^\phi(s_k^{t+1})}{2}$ ;
14    Update critic parameter  $\phi$ :  $\phi \leftarrow \arg \min_\phi \frac{1}{N} \sum_{k \in B} \|y_k - L^\phi(s_k^t)\|^2$ ;
15    Compute gradient approximation:  $\nabla J \approx \frac{1}{N} \sum_{k \in B} \nabla_\theta Q^\theta(s_k^t, a_k^t)$ ;
16    Update  $\theta$ :  $\theta \leftarrow \theta + \eta \nabla J$ ;
17    If  $\|\nabla J\| \leq \epsilon$ , stop (stopping criterion);
18  end
19 end
20 output: Optimal parameters for networks  $(U, C, A, M)$ , optimal policy, and
  optimal value function  $J(\theta)$ ;
```

Algorithm 1 TD3-based Portfolio Optimization with RTS

4 The Design of RiskawareTrader

In the face of increasingly complex and uncertain financial markets, portfolio optimization remains a central challenge for risk-aware investors who prioritize capital preservation alongside stable returns. This study introduces the RiskawareTrader Strategy (RTS), a deep reinforcement learning framework designed to address these dual objectives through a synergistic integration of multi-scale feature extraction, attention mechanisms, and risk-sensitive reward engineering.

As illustrated in Fig. 1, RTS comprises four interconnected components: market state representation, action space design, reward function formulation, and a policy network. Each component collaborates to decode market dynamics, enabling the model to adaptively balance risk and return in real-time.

4.1 State Representation

For risk-averse portfolio optimization, the state representation serves as the foundation for the RTS to perceive market dynamics, requiring comprehensive coverage of trend, momentum, volatility, and extreme market state signals. We formally define the state at time step t as a multi-dimensional set S_t , which integrates five technically validated indicators to capture distinct market dimensions. In this subsection, we define the state as Eq.(2).

$$S_t = \{MA_t, MACD_t, KDJ_t, BOLL_t, RSI_t\} \quad (2)$$

Specifically, Moving Average (MA) smooths price data to identify trends by computing the average of closing prices over rolling windows ($n = 5, 10, 20$), calculated as Eq.(3).

$$MA_t = \frac{1}{n} \sum_{i=t-n+1}^t p_i \quad (3)$$

In Eq.(3), p_i denotes the price at time i . MA helps identify trend reversals by comparing price movements against averaged values[21, 25].

Moving Average Convergence Divergence (MACD) tracks momentum by measuring the difference between exponential moving averages (EMAs) of varying lengths, its components (DIF, DEA, CD) are calculated as Eq.(4)-Eq.(6).

$$DIF_t = EMA_t^{12} - EMA_t^{26} \quad (4)$$

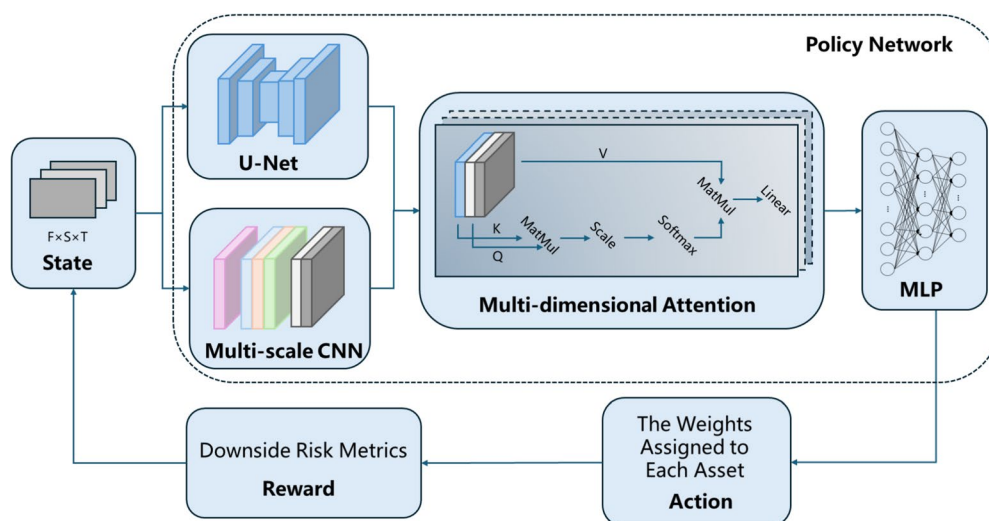


Fig. 1 The System Architecture of RTS

$$DEA_t = EMA_t^9(DIF_t) \quad (5)$$

$$MACD_t = 2 \times (DIF_t - DEA_t) \quad (6)$$

where EMA_t^n denotes the n -period EMA. This indicator highlights trend shifts and potential entry/exit points[26, 27].

Stochastic Oscillator (KDJ) identifies overbought/oversold states. For the i -th asset, its calculation is Eq.(7)-Eq.(10).

$$RSV_t = \frac{C_t - L_n}{H_n - L_n} \times 100 \quad (7)$$

where C_t is the current price, L_n is the lowest price within n days, and H_n is the highest price within n days (usually $n = 9$).

The K value:

$$K_t = \frac{2}{3}K_{t-1} + \frac{1}{3}RSV_t \quad (K_0 = 50) \quad (8)$$

The D value:

$$D_t = \frac{2}{3}D_{t-1} + \frac{1}{3}K_t \quad (D_0 = 50) \quad (9)$$

The J value:

$$J_t = 3K_t - 2D_t \quad (10)$$

Bollinger Bands (BOLL) measures volatility by defining price bands around a 20-day MA, it is calculated as Eq.(11).

$$BOLL_t = MA_t \pm 2 \times \sigma_t \quad (11)$$

where σ_t is the rolling standard deviation. Price breaches of the upper/lower bands signal potential reversals or trend continuations, aiding risk management during volatile periods[26].

Relative Strength Index (RSI) evaluates momentum by comparing average gains and losses over a period (typically 14 days), it is Eq.(12).

$$RSI_t = 100 - \frac{100}{1 + RS_t} \quad (12)$$

Where RS_t is the relative strength, calculated as the average of upward price changes divided by the average of downward price changes over a specified period. Values above 70 or below 30 indicate overbought or oversold conditions, respectively, guiding investors to avoid impulsive decisions in extreme market states[27, 28].

4.2 Action Space Design

Here, the action space is defined as a continuous vector $a_t = (w_1, w_2, \dots, w_i, \dots, w_M)$, where M denotes the number of invested assets and w_i is the allocation weight for the i -th asset.

The asset weights must satisfy the capital conservation constraints as in Eq.(13).

$$\sum_{i=1}^M w_i = 1 \quad \text{and} \quad w_i \in [0, 1]. \quad (13)$$

The constraint $w_i \in [0, 1]$ avoids short-selling and leverage, adjusting for risk averter's needs, while the dynamic adjustment w_i enables real-time portfolio rebalancing.

4.3 Reward Function Formulation

To align with the core objectives of capital preservation and stable returns for risk averter, the reward function in the model is designed to dynamically adjust incentives and penalties based on the daily portfolio return rate r_t relative to a preset target return r_{target} . It integrates two components which are return-oriented incentives $((r_t - r_{\text{target}})^2)$ and risk-oriented penalties (including P_d and D_d). P_d represents the downside risk, D_d is the penalty for consecutive underperformance. The reward function is formally defined as Eq.(14).

$$R_t = \begin{cases} \delta (r_t - r_{\text{target}})^2 + (\lambda_1 P_d + \lambda_2 D_d), & r_t < 0, \\ \alpha (r_t - r_{\text{target}})^2 + (\lambda_1 P_d + \lambda_2 D_d), & 0 \leq r_t < r_{\text{target}}, \\ \beta (r_t - r_{\text{target}})^2 + (\lambda_1 P_d + \lambda_2 D_d), & r_t \geq r_{\text{target}}, \end{cases} \quad (14)$$

where R_t denotes the total reward at time t , r_t is daily portfolio return rate. The parameters δ, α, β serve as coefficients for return-based rewards and penalties, with their relative magnitudes determining the intensity of incentives and penalties. Specifically, δ has the largest absolute value to impose a stronger penalty on negative returns ($r_t < 0$), which aligns with the risk-averse preference for avoiding losses, α has a moderate absolute value to constrain underperformance relative to the target ($0 \leq r_t < r_{\text{target}}$), and β is set as a positive value to provide a reward for excess returns ($r_t \geq r_{\text{target}}$), encouraging the model to pursue stable profits beyond the target. Meanwhile, λ_1 and λ_2 are weighting parameters used to adjust the intensity of the downside risk penalty and the consecutive underperformance penalty, respectively.

The downside risk P_d quantifies the mean squared deviation of actual returns from r_{target} when returns fall below the target, and it is calculated over a 15-day rolling window (i.e., $n = 15$) as Eq.(15). This metric specifically focuses on returns below r_{target} rather than overall volatility, which allows it to effectively capture the potential loss risk.

$$P_d = \frac{1}{n} \sum_{r_t < r_{\text{target}}} (r_{\text{target}} - r_t)^2 \quad (15)$$

While P_d effectively captures single-period losses below the target return, it fails to fully characterize sustained risk factors of the portfolio, such as consecutive underperformance that can erode long-term capital. The consecutive underperformance penalty D_d is therefore introduced to complement P_d . It is defined as $D_d = d^2$, where d is the number of consecutive days with returns below r_{target} . The squared term amplifies the penalty as the number of consecutive underperforming days increases, further restraining sustained losses. When combined with P_d , these two metrics better align with the dual needs of risk averter for capital preservation and stable returns, improving the accuracy of risk characterization.

The specific settings of hyperparameters in this reward function (including $\delta, \alpha, \beta, \lambda_1, \lambda_2$) are provided in Sect. 6.

4.4 Policy Network Architecture

As the core decision-making component of the RiskawareTrader Strategy (RTS), the policy network is designed to process high-dimensional financial data and output optimal portfolio weights. Its overall architecture, illustrated as policy network in Fig. 1, integrates four interconnected modules, including Multi-Scale Convolutional Neural Network (MSCNN), UNet Encoder-Decoder, Multi-Dimensional Attention (MDA), and Multi-Layer Perceptron (MLP). MSCNN first extracts multi-scale local features from raw data, UNet preserves contextual dependencies to enhance long-range pattern capture, MDA focuses on critical signals across feature, asset, and time dimensions, and MLP finally maps fused features to feasible asset allocation weights. This end-to-end design ensures the network balances feature representation ability and decision-making interpretability, aligning with the risk-aware goal of stable returns.

First, the MSCNN module is introduced to address the limitation of single-scale feature extraction in capturing complex price dynamics. It processes input data $X \in \mathbb{R}^{F \times S \times T}$, where $F = 12$ denotes technical features, $S = 10$

represents the number of assets, and $T = 15$ is the time step window, and using three convolutional kernel sizes (3×3 , 5×5 , 7×7) to capture multi-resolution patterns, as shown in Fig. 2. For each convolutional layer i , the operation is defined as Eq.(16)

$$y_{i,p,q,c} = \text{ReLU} \left(\sum_{m=0}^{k_i-1} \sum_{n=0}^{k_i-1} w_{i,m,n,c} x_{p+m,q+n,c} + b_i \right) \quad (16)$$

where k_i denotes the kernel size of layer i , $w_{i,m,n,c}$ is the convolutional weight for the c -th channel at position (m, n) , $x_{p+m,q+n,c}$ is the input feature value at position $(p+m, q+n)$ of the c -th channel, and b_i is the bias term. After convolution, feature maps from the three scales are concatenated and passed through a residual block to form the output X_{MSCNN} , which retains hierarchical information of price movements.

The UNet Encoder-Decoder module is integrated to mitigate the problem of contextual information loss during feature downsampling, a common issue in processing long financial time series. As depicted in Fig. 3, UNet consists of an encoder and a decoder connected by skip connections: the encoder reduces spatial dimensions (via strided convolutions) while increasing channel depth to extract high-level abstract features, and the decoder recovers spatial resolution (via transposed convolutions) to refine feature details. For an encoder input X^e , the encoding step is defined as Eq.(17).

$$X^{(e+1)} = \text{ReLU}(W^e X^e + b^e) \quad (17)$$

where W^e and b^e are the weight matrix and bias of the encoder layer, respectively. During decoding, the upsampled feature map (from the decoder input X^d with upsampling factor f) is concatenated with the corresponding encoder feature map (denoted as $X^{(e-l)}$, where l is the layer offset) to preserve low-level details, and the decoding step is defined as Eq.(18).

$$X^{(d+1)} = \text{ReLU} \left(W^d \left(\text{UpSample}(X^d, f) \oplus X^{(e-l)} \right) + b^d \right) \quad (18)$$

Fig. 2 Architecture of the Multi-Scale CNN (MSCNN)

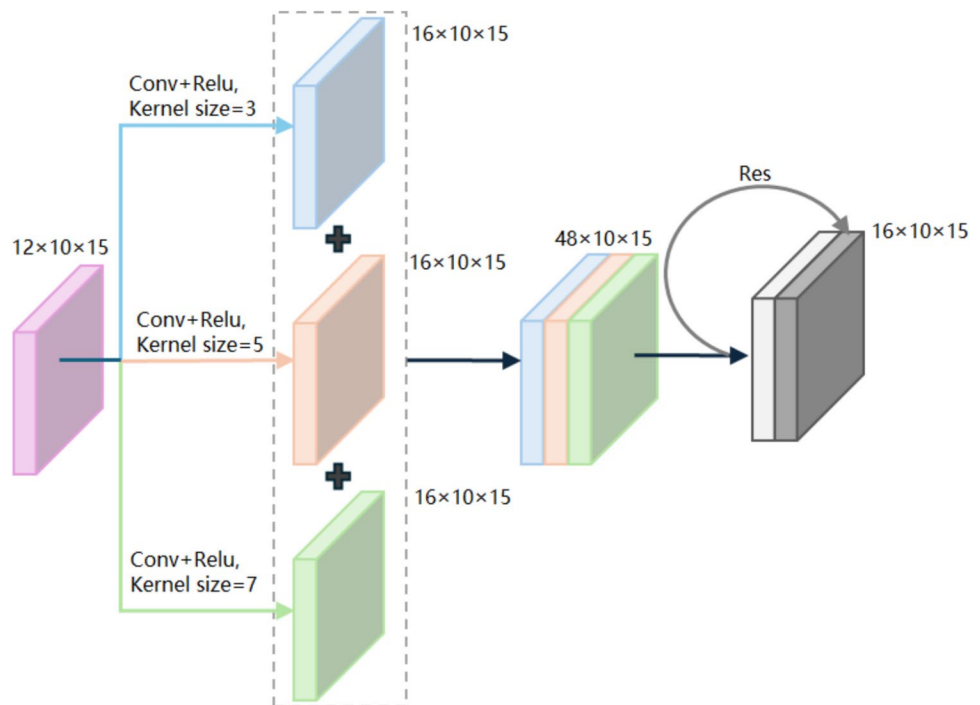


Fig. 3 Architecture of the UNet Encoder-Decoder

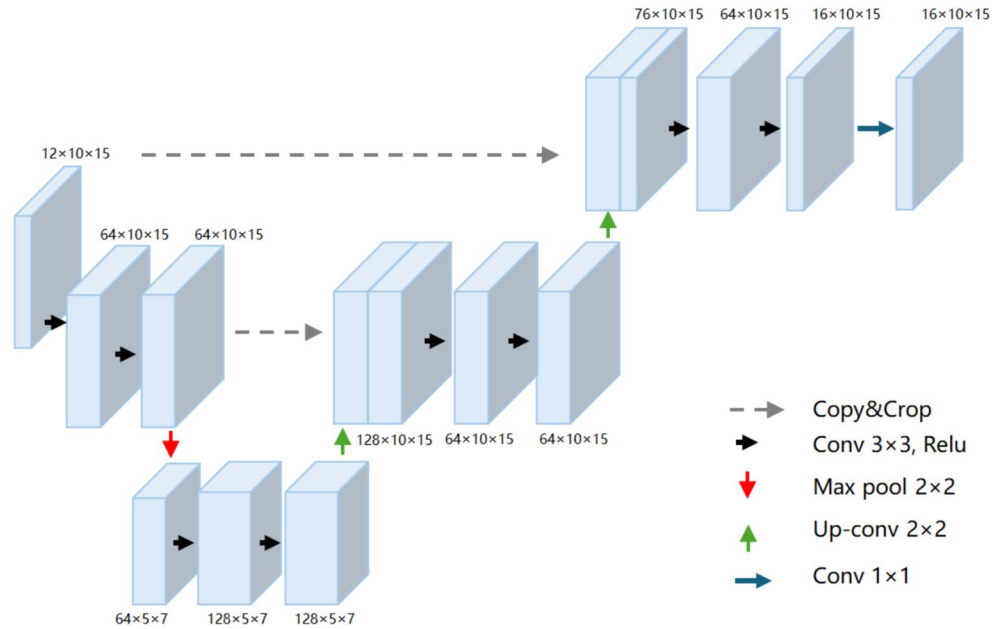
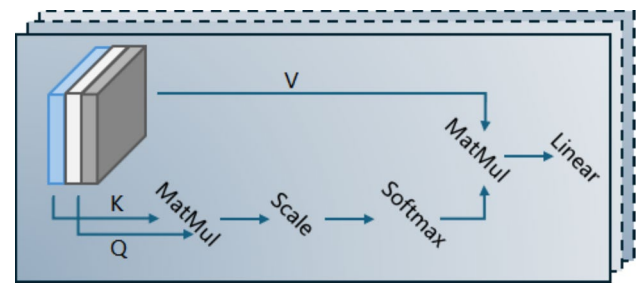


Fig. 4 Architecture of the Multi-Dimensional Attention (MDA)



Here, $\oplus$ represents concatenation, W^d and b^d are the weight and bias of the decoder layer, and this design effectively enhances the network's ability to capture long-range dependencies in time-series data.

The MDA module is designed to address the challenge of information redundancy in fused features (from MSCNN and UNet), ensuring the network focuses on critical signals for portfolio decisions. As shown in Fig. 4, MDA first fuses the outputs of MSCNN and UNet into $x^f = \text{Concat}(X_{\text{MSCNN}}, X_{\text{UNet}})$, then computes attention weights across three dimensions (feature, asset, time) in parallel. Taking the feature dimension as an example: query (Q^f), key (K^f), and value (V^f) matrices are derived via linear transformations, which are calculated as Eq.(19)-Eq.(21).

$$Q^f = x^f W_Q^f + b_Q^f \quad (19)$$

$$K^f = x^f W_K^f + b_K^f \quad (20)$$

$$V^f = x^f W_V^f + b_V^f \quad (21)$$

where W_Q^f, W_K^f, W_V^f are linear transformation weights and b_Q^f, b_K^f, b_V^f are biases. Attention scores are calculated as $S^f = (Q^f)^T K^f$, normalized by softmax to get weights $A^f = \text{softmax}(S^f)$, and applied to values to generate the feature-focused output $O^f = V^f A^f$. Parallel operations on the asset and time dimensions yield O^s (asset-focused output) and O^t (time-focused output), which are averaged to form the final MDA output X_{MDA} .

Table 1 Selected Stocks from CSI300 and DJIA Indices

CSI300 Stocks	DJIA Stocks
Ping An Insurance (Group) Company of China, Ltd	The Boeing Company
China Merchants Bank Co., Ltd	The Walt Disney Company
Kweichow Moutai Co., Ltd	Johnson & Johnson
Poly Developments and Holdings Group Co., Ltd	Walmart Inc
Sichuan Changhong Electric Co., Ltd	McDonald's Corporation
Huaneng Power International, Inc	American Express Company
Jiangsu Hengrui Medicine Co., Ltd	Visa Inc
Shengyi Electronics Co., Ltd	The Travelers Companies Inc
Luxshare Precision Industry Co., Ltd	Apple Inc
BYD Co., Ltd	The Procter & Gamble Company

Table 2 Division of Time Horizons for Training, Validation and Test Sets

Dataset	Training set	Validation set	Test set
CSI300	2016/09-2021/08	2021/09-2022/08	2022/09-2023/08 (Down)
CSI300	2014/02-2019/01	2019/02-2020/01	2020/02-2021/01 (Up)
DJIA	2016/01-2020/12	2021/01-2021/12	2022/01-2022/12 (Down)
DJIA	2015/09-2019/08	2019/09-2020/08	2020/09-2021/08 (Up)

Finally, the MLP module converts abstract features from MDA into actionable portfolio weights. It first flattens X_{MDA} into a 1D vector X_{flat} , then processes the vector through two ReLU-activated fully connected layers to learn complex feature-to-weight mappings. The output layer uses a Softmax function to ensure weights sum to 1, with the operation defined as Eq.(22).

$$a_t = \text{Softmax}(W_2 \cdot \text{ReLU}(W_1 \cdot X_{\text{flat}} + b_1) + b_2) \quad (22)$$

where W_1, W_2 are the weights of the two fully connected layers, b_1, b_2 are biases, and a_t is the final asset weight vector. This design balances adaptive decision-making (via deep feature learning) and practical feasibility (via weight normalization), ensuring the output aligns with real-world investment constraints.

5 Experiments

The experiments were conducted on a system equipped with an Intel Core i9-13900 H processor and an NVIDIA RTX 4060 GPU, running Windows 11. The software environment used Python 3.11.4, with the PyTorch 2.3.1 framework supporting CUDA 11.8 and Stable Baselines3 version 1.6.2.

To ensure the robustness and reliability of our experimental results, we meticulously calibrated the hyperparameters. For the DJIA market, the risk-free rate of return was set at 1.6575%, and our annualized target rate of return, based on our preferences with a 5.5% premium, is 7.1575%. In the CSI300 market, the risk-free rate is 3.037%, and the annualized target rate with the same premium is 8.537%. Additionally, a 0.03% commission and 0.1% slippage were incorporated to simulate real trading environments more accurately.

5.1 Datasets

This study uses four datasets from the Dow Jones Industrial Average (DJIA) Index and the China Securities Index 300 (CSI300) for comprehensive experiments. The CSI300 data is sourced from Tushare, and the DJIA data from Yahoo Finance. For each of the four datasets, 10 stocks from diverse sectors are carefully selected to ensure sectoral representation. Table 1 lists the full names of the 10 stocks chosen from each index.

The datasets are split into training, validation, and test sets as detailed in Table 2, with training periods spanning 4-5 years. The test sets for each index covers both upward and downward trends. This long training window covers diverse market scenarios (e.g., policy changes, shocks) to enhance the model's generalization.

5.2 Comparative Methods

To evaluate the performance of the RTS, this study makes a comparison between it and seven benchmark models: CRP (Constant Rebalanced Portfolio)[29], CORN (Correlation-Driven Nonparametric Learning)[30], EG (Exponential Gradient)[31], Index (the performance of the dataset index), Margin-Trader[32], ES (Ensemble Strategy) [28], and MASA-dc (Multi-Agent Self-Adaptive Framework)[33].

CRP is a static strategy model that adopts a fixed asset allocation and rebalances the portfolio daily. In our study, we use the 1/N strategy for CRP, evenly distributing funds among 10 assets without considering market changes or asset traits. CORN adjusts the portfolio based on asset correlations, while EG is an online portfolio optimization algorithm that adjusts asset weights via an exponential rule based on historical asset returns. The Index represents the performance of the dataset itself. Margin Trader is an innovative RL framework for stock margin trading that integrates margin accounts and constraints. The ES is a deep reinforcement learning based approach that combines PPO, A2C, and DDPG algorithms, selects the best performing agent based on the Sharpe ratio to optimize stock trading strategies. MASA-dc is a multi-agent and self-adaptive framework that uses a TD3-based RL-agent and a solver-based agent with the help of a market observer to balance portfolio returns and risks in volatile financial markets.

Additionally, we conducted ablation experiments to investigate the contribution of each model component, with three variant models: Only-MSCNN, Only-UNet, and Only-MDA.

5.3 Evaluation Criteria

For risk-aware investors, the robustness and certainty of investment decisions are of paramount importance. Therefore, we carefully selected multiple key performance indicators to evaluate the performance of RTS, with a focus on aspects crucial to risk-aware investors:

Annual Return (AR) is the average return rate of the portfolio over a one-year period, which is computed as Eq.(23).

$$AR = \left(1 + \frac{E_{\text{end}} - E_{\text{start}}}{E_{\text{start}}} \right)^{\frac{252}{n-1}} - 1 \quad (23)$$

where E_{start} is the value of the investment at the start, E_{end} is the value at the end, and n is the total number of trading days in the investment period. A stable and positive annual return is key for risk-aware investors when evaluating the effectiveness of the model in long-term investment scenarios.

Maximum Drawdown (MDD) is the maximum loss from a peak to a trough during the investment period. Its calculation formula is as Eq.(24).

$$MDD = \frac{P_{\text{max}} - P_{\text{min}}}{P_{\text{max}}} \quad (24)$$

where P_{max} is the highest portfolio value and P_{min} is the lowest value after P_{max} . A lower MDD indicates greater portfolio stability and better risk control.

Sharpe Ratio (SR) assesses the excess return earned per unit of risk. The formula is Eq.(25).

$$SR = \frac{R_p - R_f}{\sigma_p} \quad (25)$$

where R_p is the average return of the portfolio, R_f is the risk-free rate (typically approximated by government bond yields), and σ_p is the standard deviation of the portfolio returns. A higher Sharpe Ratio indicates a better risk-return tradeoff.

Calmar Ratio (CR) measures the risk-adjusted return, calculated as the annualized return divided by the maximum drawdown. The formula is Eq.(26).

$$\text{Calmar Ratio} = \frac{R_p}{MDD} \quad (26)$$

A higher Calmar Ratio indicates the ability to achieve higher returns while managing risk effectively, especially after experiencing large drawdowns.

Sterling Ratio (STR) evaluates the portfolio's return relative to its drawdowns. It is typically defined as Eq.(27).

$$\text{Sterling Ratio} = \frac{R_p - R_f}{MDD_{avg}} \quad (27)$$

where R_p is the portfolio's return, R_f is the risk-free rate, and MDD_{avg} represents the average maximum drawdown beyond a certain threshold. A higher Sterling Ratio indicates that the portfolio delivers better returns after accounting for significant drawdowns.

Volatility (Vol) quantifies the degree of fluctuation in the portfolio's returns. It is calculated as the standard deviation of the portfolio returns over a period, defined as Eq.(28).

$$\text{Volatility} = \sqrt{\frac{1}{T-1} \sum_{t=1}^T (r_t - R_p)^2} \quad (28)$$

Lower volatility indicates more stable returns and less risk.

Negative Return Days (NRD) calculates the number of days the portfolio has had negative returns. A lower count suggests stronger sustained profitability and better performance consistency.

Days Below Target Return (DBTR) counts the number of days the portfolio's return falls below a preset target. A lower value signifies that the model has a higher likelihood of achieving the desired returns consistently.

5.4 Performance Analysis

This study conducts a comprehensive evaluation of various static strategies, online learning algorithms, and reinforcement learning models on the DJIA and CSI300 datasets. The focus is on analyzing the proposed RTS and its extended version RTS with Cash, and comparing them with benchmark strategies.

Table 3 Performance Metrics on the Test Set of CSI300 (Down)

Models	AR↑	MDD↓	SR↑	CR↑	STR↑	Vol↓	NRD↓	DBTR↓
CRP	0.0316	0.1114	0.0893	0.2840	0.0113	0.1700	121	125
CORN	0.0176	0.1140	0.0046	0.1542	-0.1122	0.1657	115	117
EG	0.0246	0.1151	0.0497	0.2134	-0.0504	0.2418	123	128
Index	-0.0228	0.1201	-0.2539	-0.1898	-0.4426	0.1605	124	130
Margin-Trader	0.0437	0.2466	0.1905	0.1774	0.0542	0.2988	112	113
ES	0.0128	0.1274	-0.0432	0.1006	-0.1377	0.1497	120	140
MASA-dc	-0.0694	0.1362	-0.4777	-0.5097	-0.7326	0.1803	129	134
Only-MSCNN	0.0939	0.1078	0.4334	0.8712	0.5894	0.1706	119	125
Only-UNet	0.0935	0.1082	0.4317	0.8641	0.5833	0.1701	122	129
Only-MDA	0.1045	0.1239	0.4884	0.8431	0.5980	0.1712	119	124
RTS	0.1112	0.0968	0.5196	1.1490	0.8353	0.1735	114	118
RTS with Cash	0.0957	0.1144	0.4677	0.8368	0.5712	0.1567	118	124

Table 4 Performance Metrics on the Test Set of CSI300 (Up)

Models	AR↑	MDD↓	SR↑	CR↑	STR↑	Vol↓	NRD↓	DBTR↓
CRP	0.2015	0.1741	0.7791	1.1573	0.9829	0.2315	112	118
CORN	0.2402	0.1960	0.8958	1.2260	1.0710	0.2389	109	113
EG	0.2107	0.1711	0.8164	1.2318	1.0543	0.3142	114	117
Index	0.3267	0.1608	1.2730	2.0321	1.8432	0.2173	101	107
Margin-Trader	0.1216	0.1408	0.4369	0.8636	0.6479	0.2903	116	111
ES	0.1074	0.1409	0.4391	0.7618	0.5463	0.2176	111	115
MASA-dc	0.2204	0.1812	0.8084	1.2164	1.0488	0.2465	115	119
Only-MSCNN	0.4485	0.1841	1.5123	2.4367	2.2717	0.2450	106	110
Only-UNet	0.4487	0.1842	1.5132	2.4352	2.2704	0.2449	106	112
Only-MDA	0.3138	0.1670	1.1757	1.8786	1.6967	0.2287	112	115
RTS	0.4493	0.1842	1.5163	2.4398	2.2748	0.2447	106	110
RTS with Cash	0.4084	0.1698	1.5051	2.4050	2.2262	0.2243	106	111

Table 5 Performance Metrics on the Test Set of DJIA (Down)

Models	AR↑	MDD↓	SR↑	CR↑	STR↑	Vol↓	NRD↓	DBTR↓
CRP	−0.1521	0.2795	−0.7286	−0.5440	−0.6033	0.2173	127	131
CORN	−0.1804	0.3036	−0.8954	−0.5942	−0.6488	0.2152	121	123
EG	−0.1508	0.2786	−0.7173	−0.5413	−0.6008	0.1829	127	133
Index	−0.0356	0.1969	−0.1604	−0.1810	−0.2652	0.2026	119	125
Margin-Trader	−0.1266	0.1559	−1.0413	−0.8120	−0.9183	0.1372	124	126
ES	−0.0803	0.3450	−0.1906	−0.2327	−0.2808	0.2967	119	123
MASA-dc	−0.0844	0.2440	−0.3865	−0.3459	−0.4138	0.2125	117	121
Only-MSCNN	−0.0653	0.2375	−0.2735	−0.2748	−0.3446	0.2193	117	120
Only-UNet	−0.0821	0.2618	−0.3284	−0.3137	−0.3770	0.2304	121	126
Only-MDA	−0.0656	0.2317	−0.2921	−0.2831	−0.3546	0.2120	116	119
RTS	−0.0649	0.2376	−0.2725	−0.2733	−0.3430	0.2191	115	118
RTS with Cash	−0.0626	0.2106	−0.3260	−0.2971	−0.3758	0.1922	115	117

Table 6 Performance Metrics on the Test Set of DJIA (Up)

Models	AR↑	MDD↓	SR↑	CR↑	STR↑	Vol↓	NRD↓	DBTR↓
CRP	0.2672	0.0937	1.4609	2.8507	2.6738	0.1599	109	113
CORN	0.1873	0.1019	1.1049	1.8381	1.6754	0.1510	114	117
EG	0.2637	0.0941	1.4372	2.8032	2.6270	0.1481	110	115
Index	0.3242	0.0810	2.0114	4.0018	3.7972	0.1363	98	102
Margin-Trader	0.1993	0.2397	0.8666	0.8318	0.7626	0.2190	114	122
ES	0.2027	0.1130	1.1038	1.7932	1.6465	0.1648	115	137
MASA-dc	0.2939	0.0874	1.6541	3.3643	3.1745	0.1529	108	113
Only-MSCNN	0.3448	0.0996	1.7951	3.4607	3.2943	0.1633	110	115
Only-UNet	0.3784	0.0936	1.8192	4.0409	3.8639	0.1759	109	113
Only-MDA	0.3440	0.0998	1.7931	3.4473	3.2812	0.1632	113	116
RTS	0.3804	0.0922	1.9475	4.1235	3.9438	0.1640	105	110
RTS with Cash	0.3435	0.0860	1.8097	3.9930	3.8003	0.1613	109	114

The experimental results, as shown in Tables 3, 4, 5, and 6, demonstrate the adaptability and effectiveness of the RTS across varying market conditions.

On the CSI300 downward market (Table 3), RTS achieved an annual return (AR) of 0.1112, outperforming all benchmark models, including Margin-Trader (0.0437) and ES (0.0128). This superior return indicates that RTS effectively identifies profitable opportunities even in declining markets. Additionally, RTS recorded the lowest maximum drawdown (MDD) at 0.0968, substantially lower than MASA-dc (0.1362) and ES (0.1274),

showcasing its ability to mitigate extreme losses. The high Sharpe ratio (SR) of 0.5196 and Calmar ratio (CR) of 1.1490 further validate the model's strong risk-adjusted returns, aligning with its objective of balancing return maximization with downside risk control. Notably, ablation studies confirm the contribution of each network component, with the full RTS model outperforming its individual variants, Only-MSCNN and Only-UNet, in both return and risk control metrics.

In the CSI300 upward market (Table 4), RTS delivered an AR of 0.4493, comparable to Only-MSCNN (0.4485) and Only-UNet (0.4487), while significantly surpassing traditional strategies such as CRP (0.2015) and EG (0.2107). The model maintains a relatively low MDD (0.1842), ensuring resilience in bullish conditions. The high Sharpe ratio (1.5163) and Sterling ratio (2.2748) further underscore RTS's capability to generate stable excess returns while managing risk exposure effectively. Compared to reinforcement learning-based models such as Margin-Trader (0.1216 AR, 0.1408 MDD), RTS demonstrates superior adaptability in capturing upward trends while maintaining disciplined drawdown management.

For the DJIA downward market (Table 5), RTS underperforms relative to the Index, achieving an AR of -0.0649 compared to the Index's -0.0356 . This discrepancy likely stems from the Index representing the overall market, while RTS operates on a subset of 10 stocks, limiting its ability to track broader market movements. Furthermore, heightened market volatility during downturns may present challenges in maintaining portfolio stability. Despite this, RTS achieves a relatively moderate MDD (0.2376), outperforming traditional methods such as EG (0.2786) and CRP (0.2795), and exhibits a lower number of negative return days (115), indicating controlled loss exposure. The RTS with Cash variant further reduces MDD to 0.2106, highlighting the effectiveness of incorporating cash assets for enhanced risk management in adverse market conditions.

In the DJIA upward market (Table 6), RTS performs remarkably, with an annual return of 0.3804, outperforming all benchmark models except the Index in all aspects. Despite the strong performance of the Index, RTS demonstrates superior risk-adjusted returns, reflected in its high Sharpe ratio (1.9475) and Calmar ratio (4.1235). Compared to other reinforcement learning models, such as MASA-dc (0.2939 AR, 0.0874 MDD), RTS exhibits greater efficiency in maximizing returns while maintaining similar risk exposure. Ablation study results confirm

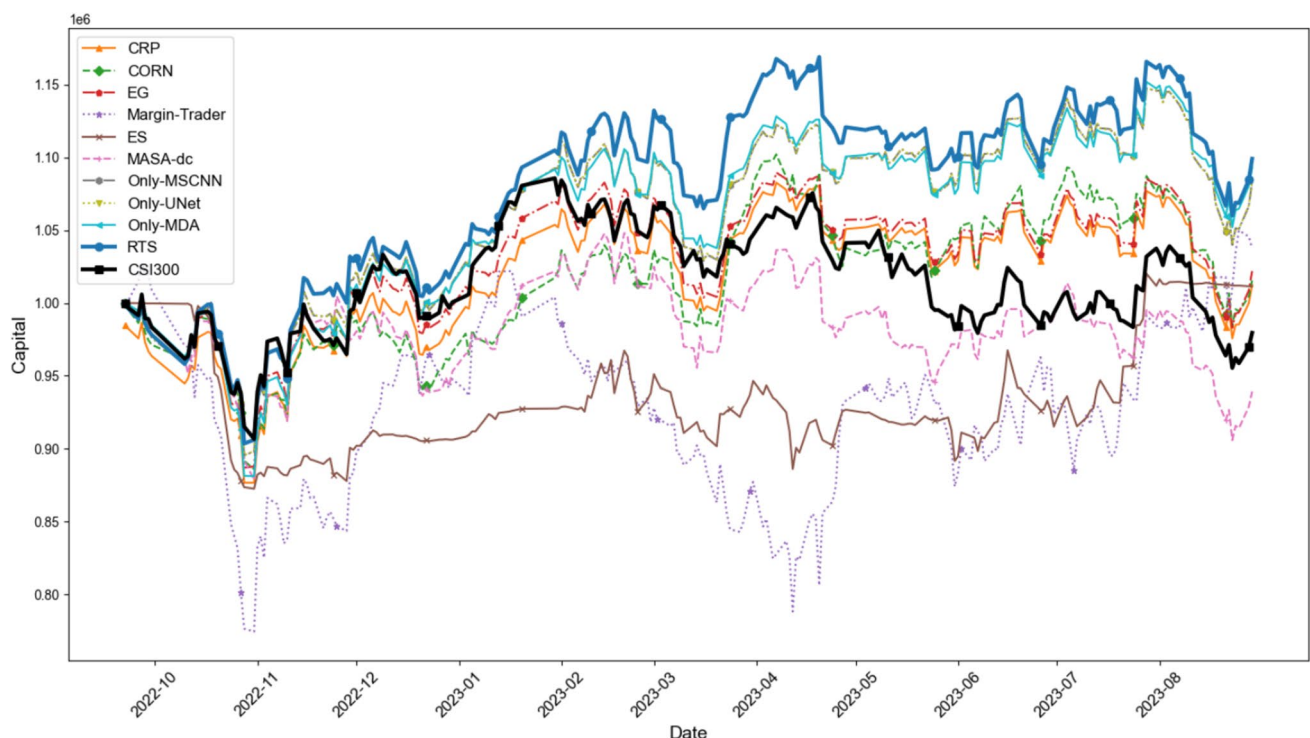


Fig. 5 Portfolio Value Comparison on the Test Set of CSI300 (Down)

the significance of the MSCNN, UNet, and attention mechanisms, which collectively enhance RTS's adaptability to bullish market conditions.

As depicted in Fig. 5, which showcases the portfolio value comparison for the CSI300 during a downward-trending market period, a multitude of lines representing diverse trading strategies are presented. Among these, the black bold line (CSI300 index curve) explicitly indicates that the entire CSI300 test set market is in a downward trend, which serves as the benchmark for defining the downward market in this study. The dark blue line, corresponding to the RTS, is particularly notable. Amidst the overall challenging market conditions, as evidenced by the downward-sloping trajectory of the CSI300 index (depicted by the black line), RTS demonstrates the ability to sustain a relatively stable upward-moving capital value. The fluctuations of RTS are less erratic compared to several other strategies; for instance, the Margin-Trader, whose capital value experiences substantial declines. Strategies such as the CRP, exhibit a more stable but rather sluggish growth pattern, failing to capitalize on the growth potential that RTS effectively harnesses within this adverse market environment.

As depicted in Fig. 6, which focuses on the portfolio value comparison during an upward-market phase of the DJIA, the growth trajectories of the asset values for various strategies are visualized. Here, the black bold line (DJIA index curve) clearly signifies that the entire DJIA test set market is in an upward trend, which is the basis for defining the upward market in this study. The RTS, denoted by the dark blue line, demonstrates a consistent and robust upward trend. In stark contrast, although the CRP maintains relatively stable growth with low volatility, its growth rate is significantly slower than that of RTS. The Margin-Trader exhibits a highly volatile and complex curve, and lags far behind RTS in terms of overall upward growth. This graphical representation unequivocally underscores that RTS is more proficient in seizing upward market opportunities and achieving superior capital appreciation compared to other strategies within the context of the DJIA upward-market scenario.

Furthermore, as shown in Fig. 7 and Fig. 8, the trend in the Reward Sum during training further demonstrates the stability of RTS's learning process. On both the DJIA and CSI300 training sets, RTS's Reward Sum fluctuated less and remained at a higher level, indicating that the model could effectively learn the complex characteristics of the market and continuously improve its strategy performance over multiple training rounds. In contrast,

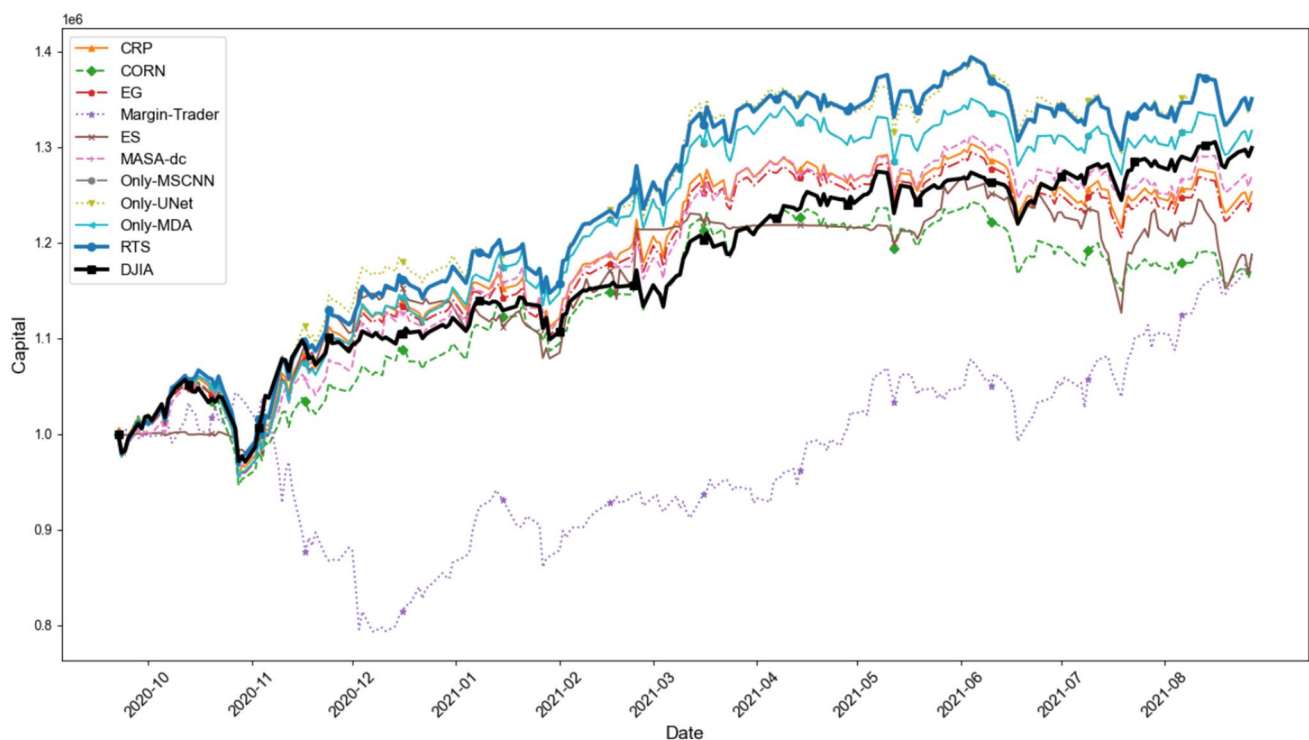


Fig. 6 Portfolio Value Comparison on the Test Set of DJIA (Up)

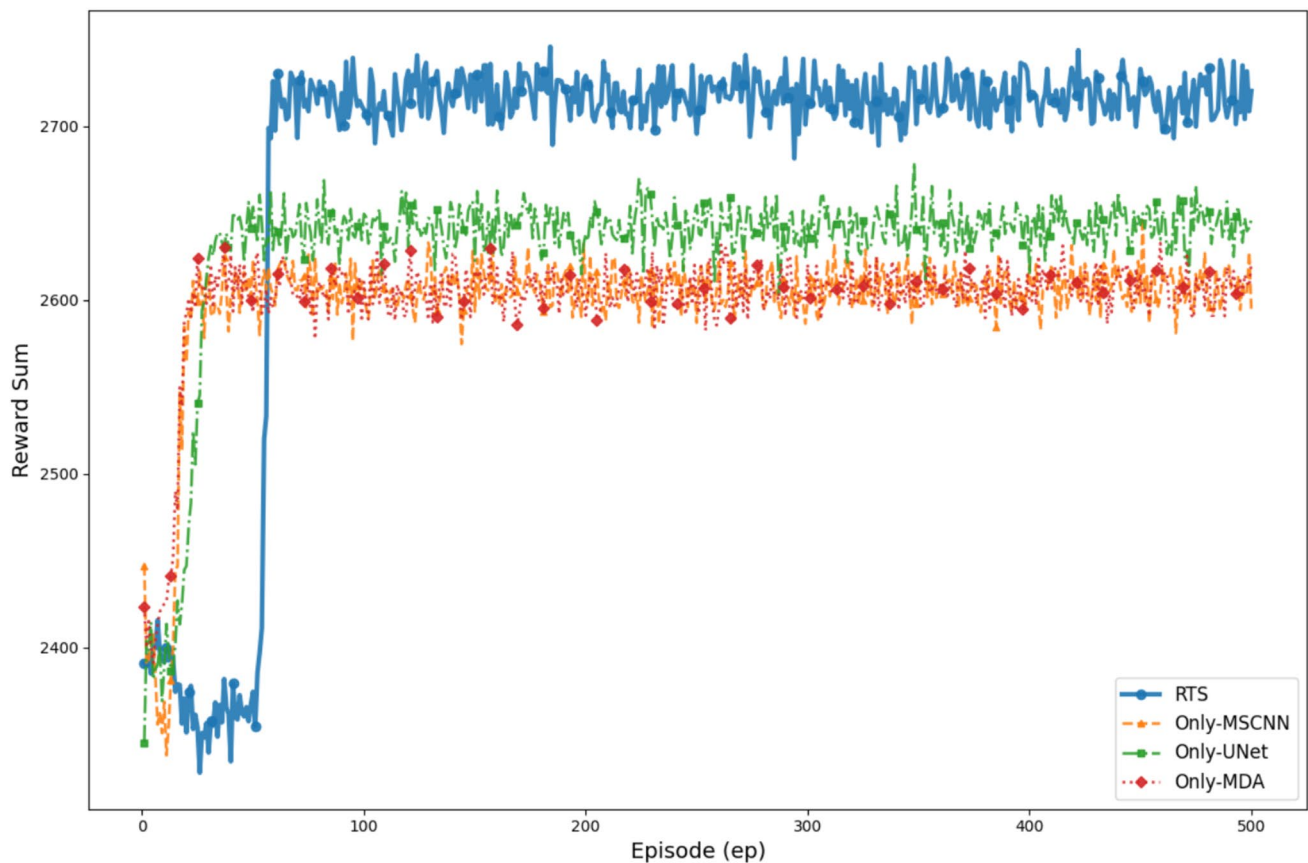


Fig. 7 Reward Trend During Training on the DJIA (Up)

other models such as Only-MSCNN, Only-UNet, and Only-MDA exhibited more significant fluctuations in their Reward Sum and had lower overall levels, reflecting their instability during the strategy learning process.

From the comparison in Table 7, RTS demonstrated high efficiency in inference time, achieving 0.0056 s on the DJIA dataset and 0.0058 s on the CSI300 dataset. These results are significantly faster than complex models like Margin-Trader (0.5300 s) and ES (0.2400 s). This highlights that RTS balances low inference time with excellent performance in returns and risk control, making it well-suited for real-time investment decision-making.

In summary, the experimental results show that both RTS and RTS with Cash outperform traditional mean-variance models and benchmark strategies on the DJIA and CSI300 datasets, particularly in high-volatility market environments where they demonstrate strong return capabilities and risk control. RTS excels in asset allocation and reward adjustment, exhibiting strong adaptability and stability. Furthermore, RTS has a significant advantage in inference efficiency and practical application. Therefore, this study not only provides a new perspective for portfolio optimization but also offers valuable insights for efficient and robust investment decision-making.

6 Discussion

Hyperparameters are critical to the performance of the RiskawareTrader Strategy (RTS), and their settings balance practicality, efficiency, and alignment with risk-averse objectives.

For the number of invested assets $M = 10$, this choice strikes a balance between portfolio diversification and computational efficiency. Notably, RTS is not limited to $M = 10$, it can be flexibly extended to any positive integer M by adjusting the output dimension of the Multi-Layer Perceptron (MLP) module. However, increasing M

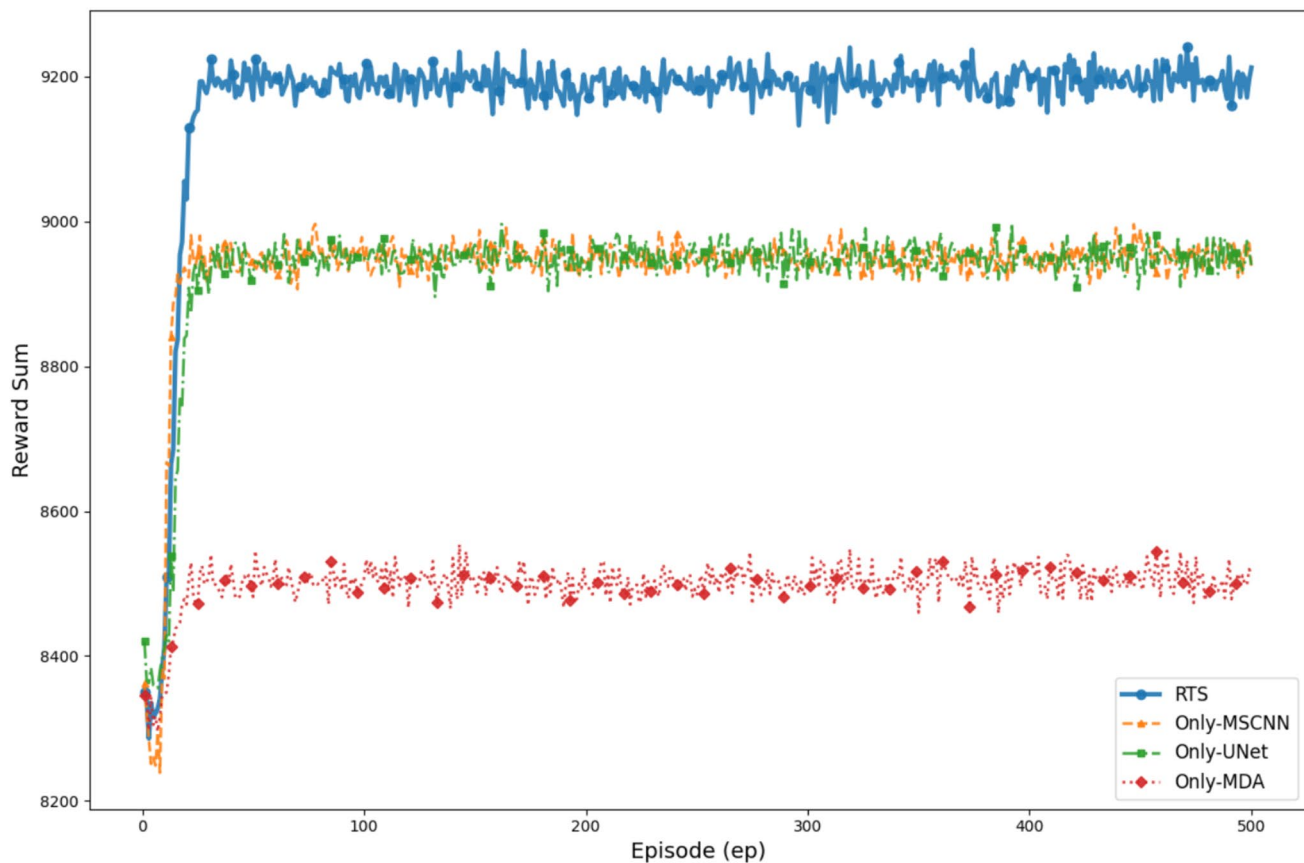


Fig. 8 Reward Trend During Training on the CSI300 (Down)

Table 7 Inference Time of Different Models on the DJIA (Up) and CSI300 (Down)

Models	DJIA (Up) (s)↓	CSI300 (Down) (s)↓
Margin-Trader	0.5300	0.4900
ES	0.2400	0.2300
MASA-dc	0.0166	0.0124
Only-MSCNN	0.0033	0.0041
Only-UNet	0.0053	0.0039
Only-MDA	0.0030	0.0027
RTS	0.0056	0.0058
RTS with Cash	0.0048	0.0047

expands the strategy space, which prolongs training time, and future work could address this via parallel computing or model compression.

In terms of dataset and time window design, the 4-5 year training period for both CSI300 and DJIA datasets ensures coverage of diverse market scenarios such as policy shifts and economic shocks. In terms of the time window, compared with the 5 day window [34], 31 day window [33] and 50 day window [13] used in related studies, this study adopts a 15 day window. This window falls in the middle range, and it focuses on extracting short-term sequential patterns and conducting experimental validation.

The hyperparameters of the reward function in the model were determined based on empirical insights. Specifically, $\delta = -100000$ is set to a relatively large value to strengthen the penalty for negative returns, while $\alpha = -10000$ and $\beta = 10000$ are used to balance the situations of returns below the target and excess returns, respectively, and $\lambda_1 = -10000$ prioritizes minimizing downside risk and $\lambda_2 = -0.01$ penalizes consecutive underperformance.

7 Conclusion

This study introduces a RiskawareTrader Strategy (RTS) leveraging deep reinforcement learning, which could offer a novel approach to portfolio optimization for risk-aware investors. By integrating multi-scale convolutional networks, UNet architecture, and multi-dimensional attention mechanisms, RTS may enable effective extraction of hierarchical features from financial market data, while a downside risk-sensitive reward function could help balance return-seeking and risk control objectives.

Empirical results across varied market conditions, including upward trends in the DJIA and downward phases in the CSI 300, suggest that RTS demonstrates promising performance. The strategy achieved annualized returns of up to 44.93% in bullish scenarios and maintained maximum drawdowns below 10% in bearish environments, which may reflect its capacity to adapt to different market regimes. Risk-adjusted metrics, such as the Sharpe ratio and Calmar ratio, indicate that RTS could potentially provide stable risk-return tradeoffs, though further validation in extended market cycles may be warranted.

Ablation studies imply that the combination of MSCNN, UNet, and MDA components may contribute to RTS's performance, although their relative importance could vary under different market conditions. The model's relatively low volatility and fewer negative return days might enhance investor confidence in long-term applications, while its robustness in extreme market events remains to be tested.

Future research could explore enhancing RTS's resilience to extreme market shocks through advanced regularization techniques. Additionally, incorporating diverse data sources such as alternative data or macroeconomic indicators may improve the model's generalization ability. These efforts might further refine RTS as a viable tool for risk-aware portfolio optimization, with acknowledgment of the need for continuous adaptation to evolving market dynamics.

Author Contributions Min Yang designed the research framework and conducted the experiments. Yi Hu was responsible for data preprocessing and model implementation. Jin Wang supervised the study, provided critical insights, and contributed to the manuscript revision.

Funding Not applicable.

Data Availability The DJIA data used in this study was obtained from Yahoo Finance. You could access historical data from Yahoo Finance at <https://finance.yahoo.com/>. The CSI300 data was sourced from Tushare. To access relevant data, you could visit the Tushare official website at <https://tushare.pro/>.

Materials Availability Not applicable.

Code Availability Not applicable.

Declarations

Competing interests The author has no conflict of interest to declare in relation to the content of this article.

Ethics approval and consent to participate Not applicable.

Consent for publication Not applicable.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

1. Markowitz, H.M.: Portfolio selection. *J. Finance* **7**(1), 71–91 (1952)
2. Hirshleifer, D., Shumway, T.: Good day sunshine: stock returns and the weather. *J. Finance* **58**(3), 1009–1032 (2003)
3. Kehoe, T.J., Pujolas, P.S., Rossbach, J.: Quantitative trade models: developments and challenges. *Annual Rev. Econ.* **9**(1), 295–325 (2017)
4. Zhang, Y., Zhao, P., Wu, Q., Li, B., Huang, J., Tan, M.: Cost-sensitive portfolio selection via deep reinforcement learning. *IEEE Trans. Knowl. Data Eng.* **34**(1), 236–248 (2020)
5. Zhao, T., Ma, X., Li, X., Zhang, C.: Asset correlation based deep reinforcement learning for the portfolio selection. *Expert Syst. Appl.* **221**, 119707 (2023)
6. Jiang, W., et al.: New reinforcement learning based on representation transfer for portfolio management. *Knowl.-Based Syst.* **293**, 111697 (2024)
7. Vajda, I.: Analysis of semi-log-optimal investment strategies. *Prague Stochastics* (2006)
8. Agarwal, A., Hazan, E., Kale, S., et al.: Algorithms for portfolio management based on the newton method. In: Proceedings of the 23rd International Conference on Machine Learning, pp. 9–16 (2006)
9. Li, B., Hoi, S.C.H.: On-line portfolio selection with moving average reversion. *arXiv preprint [arXiv:1206.4626](https://arxiv.org/abs/1206.4626)* (2012)
10. Györfi, L., Lugosi, G., Udina, F.: Nonparametric kernel-based sequential investment strategies. *Math. Finance Int. J. Math. Stat. Financ. Econ.* **16**(2), 337–357 (2006)
11. Hester, T., Vecerik, M., Pietquin, O., et al.: Deep q-learning from demonstrations. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 32 (2018)
12. Mnih, V., Kavukcuoglu, K., Silver, D., et al.: Human-level control through deep reinforcement learning. *Nature* **518**(7540), 529–533 (2015)
13. Jiang, Z., Xu, D., Liang, J.: A deep reinforcement learning framework for the financial portfolio management problem. *arXiv preprint [arXiv:1706.10059](https://arxiv.org/abs/1706.10059)* (2017)
14. Xiong, Z., Liu, X.-Y., Zhong, S., Yang, H., Walid, A.: Practical deep reinforcement learning approach for stock trading. *arXiv preprint [arXiv:1811.07522](https://arxiv.org/abs/1811.07522)* (2018)
15. Enkhsaikhan, B., Jo, O.: Risk-averse reinforcement learning for portfolio optimization. *ICT Express* **10**(4), 857–862 (2024)
16. Enkhsaikhan, B., Jo, O.: Risk-constrained reinforcement learning with augmented lagrangian multiplier for portfolio optimization. *IEEE Trans. Big Data* **11**(5), 2489–2502 (2025)
17. Cui, T., Ding, S., Jin, H., Zhang, Y.: Portfolio constructions in cryptocurrency market: a cvar-based deep reinforcement learning approach. *Econ. Model.* **119**, 106078 (2023)
18. Yoo, S.J., Gu, Y.H., et al.: Safety aarl: weight adjustment for reinforcement-learning-based safety dynamic asset allocation strategies. *Expert Syst. Appl.* **227**, 120297 (2023)
19. Chen, T.-F., Kuang, X.-J., Liao, S.-L., Lin, S.-K.: Portfolio allocation with dynamic risk preferences via reinforcement learning. *Comput. Econ.* **64**(4), 2033–2052 (2024)
20. Li, J., Zhang, Y., Yang, X., Chen, L.: Online portfolio management via deep reinforcement learning with high-frequency data. *Inf. Process. Manag.* **60**(3), 103247 (2023)
21. Zou, J., Lou, J., Wang, B., Liu, S.: A novel deep reinforcement learning based automated stock trading system using cascaded lstm networks. *Expert Syst. Appl.* **242**, 122801 (2024). <https://doi.org/10.1016/j.eswa.2023.122801>
22. Xu, K., Zhang, Y., Ye, D., Zhao, P., Tan, M.: Relation-aware transformer for portfolio policy learning. In: Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, pp. 4647–4653 (2024)
23. Ramya, D., et al.: Reinforcement learning driven trading algorithm with optimized stock portfolio management scheme to control financial risk. *SN Comput. Sci.* **6**(1), 1–16 (2025)
24. Hambly, B., Xu, R., Yang, H.: Recent advances in reinforcement learning in finance. *Math. Financ.* **33**(3), 437–503 (2023)
25. Dash, R., Dash, P.K.: A hybrid stock trading framework integrating technical analysis with machine learning techniques. *J. Finance Data Sci.* **2**(1), 42–57 (2016). <https://doi.org/10.1016/j.jfds.2016.03.002>
26. Zhang, H., Fang, Y., Liu, X.: A deep reinforcement learning model for portfolio management incorporating historical stock prices and risk information. In: Proceedings of the 2024 8th International Conference on Deep Learning Technologies. ICDLT '24, pp. 1–8. Association for Computing Machinery, New York, NY, USA (2024). <https://doi.org/10.1145/3695719.3695720>
27. Jang, J., Seong, N.: Deep reinforcement learning for stock portfolio optimization by connecting with modern portfolio theory. *Expert Syst. Appl.* **218**, 119556 (2023). <https://doi.org/10.1016/j.eswa.2023.119556>
28. Yang, H., et al.: Deep reinforcement learning for automated stock trading: an ensemble strategy. In: Proceedings of the First ACM International Conference on AI in Finance (2020)
29. Cover, T.M.: Universal portfolios. *Math. Finance* **1**(1), 1–29 (1991)
30. Li, B., Hoi, S.C., Gopalkrishnan, V.: Corn: correlation-driven nonparametric learning approach for portfolio selection. *ACM Trans. Intell. Syst. Technol. (TIST)* **2**(3), 1–29 (2011)

31. Helmbold, D.P., Schapire, R.E., Singer, Y., Warmuth, M.K.: On-line portfolio selection using multiplicative updates. *Math. Financ.* **8**(4), 325–347 (1998)
32. Gu, J., Du, W., Rahman, A.M.M., Wang, G.: Margin trader: A reinforcement learning framework for portfolio management with margin and constraints. In: 4th ACM International Conference on AI in Finance, pp. 610–618 (2023)
33. Li, Z., Tam, V., Yeung, K.L.: Developing a multi-agent and self-adaptive framework with deep reinforcement learning for dynamic portfolio risk management. In: Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems (AAMAS '24), pp. 1174–1182 (2024)
34. Cheng, L.-C., Sun, J.-S.: Multiagent-based deep reinforcement learning framework for multi-asset adaptive trading and portfolio management. *Neurocomputing* **594**, 127800 (2024)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Min Yang^{1,2} · Jin Wang^{1,2} · Yi Hu^{1,2}

✉ Jin Wang
wangjin@cdu.edu.cn

Min Yang
ym1289645507@gmail.com

Yi Hu
huyi_028@foxmail.com

¹ College of Computer Science, Chengdu University, 2025 Chengluo Road, Chengdu 610106, China

² Sichuan Provincial University Key Laboratory of Pattern Recognition and Intelligent Information Processing, 2025 Chengluo Road, Chengdu 610106, China